

ACTIVIDAD 9

Estructuras, Métodos de Ordenación y Búsqueda

Librería Propia

Objetivos

- Evaluar los conocimientos adquiridos sobre estructuras (`struct`), métodos de ordenación y búsqueda en C.
 - Desarrollar habilidades prácticas en la implementación de algoritmos utilizando funciones personalizadas y librerías propias.
 - Fomentar el uso de buenas prácticas de programación y optimización de código.
 - Practicar la implementación de menús interactivos y operaciones con estructuras de datos.
-

Competencias a Desarrollar

- Manejo de estructuras y `typedef` en C.
 - Implementación de métodos de ordenación (Bubble Sort, Quick Sort, Merge Sort, etc.).
 - Implementación de métodos de búsqueda (secuencial y binaria).
 - Modularización del código mediante librerías propias.
 - Uso de arreglos dinámicos para la gestión eficiente de grandes volúmenes de datos.
 - Desarrollo de menús interactivos para la gestión de registros.
-

Requisitos Previos

- Conocimiento sobre estructuras en C (`struct` y `typedef`).
- Familiaridad con la implementación de algoritmos de ordenación y búsqueda.
- Habilidad para modularizar el código en archivos `.h` y `.c`.
- Experiencia en la manipulación de arreglos y punteros.
- Configuración del entorno de desarrollo con GCC o G++.
- Editor de texto configurado (VSCode).

Instrucciones

1. Abre Visual Studio Code y configura tu entorno para compilar programas en C/C++ usando GCC o G++.
2. Completa todos los ejercicios propuestos en lenguaje C.
3. Nombra cada archivo fuente con extensión .cpp según las convenciones establecidas en clase (por ejemplo, iniciales_ACT9.cpp, PNY_ACT9_01.cpp).
4. Para cada ejercicio:
 - Captura una imagen del código fuente.
 - Captura una imagen de la ejecución del programa mostrando la salida en consola.
5. Elabora un documento Word que incluya:
 - Una portada con el nombre de la actividad, tu nombre, matrícula y grupo.
 - Las capturas de pantalla del código.
 - Las capturas de pantalla de las ejecuciones.
 - Comentarios o explicaciones relevantes.
6. Exporta el documento a formato PDF.
7. Nombra el archivo PDF como INICIALES_PE_RP09.PDF.
8. Entrega en la plataforma CLASSROOM:
 - El archivo PDF del reporte de práctica.
 - El archivo PDF del anexo con capturas y código.
 - Los archivos fuente .cpp del programa y la librería creada.

Ejercicio

Menú Interactivo para Operaciones con Alumnos

Desarrolle un programa en C que implemente un menú interactivo con las siguientes opciones:

MENÚ

1. AGREGAR (AUTOMÁTICO 10 REGISTROS)
2. AGREGAR MANUAL
3. ELIMINAR REGISTRO (lógico)
4. BUSCAR
5. ORDENAR
6. IMPRIMIR
7. SALIR

El programa deberá repetirse cuantas veces lo desee el usuario y estar 100% validado.

Detalles de Implementación

1. AGREGAR (AUTOMÁTICO 10 REGISTROS)

- Agregue automáticamente 10 registros de alumnos con datos generados aleatoriamente.
- Valida que la matrícula no esté repetida antes de agregar un nuevo registro.

2. AGREGAR MANUAL

- Permite al usuario ingresar manualmente los datos de un alumno.
- Valide que los datos ingresados sean correctos (por ejemplo, matrícula única, sexo válido).

3. ELIMINAR REGISTRO (lógico)

- Implementa una función que permita eliminar un registro sin borrarlo físicamente.
- La baja lógica se implementará cambiando el campo **status** a 0 en lugar de eliminar el dato del arreglo.
- Muestra un mensaje confirmando si el registro fue eliminado correctamente o si la matrícula no existe.

4. BUSCAR

- Implementa una función para buscar a un alumno por su matrícula utilizando los métodos:
 - Búsqueda secuencial.
 - Búsqueda binaria (requiere que el arreglo esté ordenado).
- Muestra los datos completos del alumno si se encuentra. (MODO de Registro)
- El programa debe seleccionar el método según sea el estado del arreglo.

5. ORDENAR

- Ordene los registros por matrícula utilizando tres métodos diferentes:
 - Burbuja.
 - Selección.
 - Inserción.
- Optimice el rendimiento del vector según su tamaño el método mas adecuado.
- No ordenar vector si el vector ya está ordenado.

6. IMPRIMIR

- Imprima los registros activos (status = 1) en un formato claro y organizado.
- Mostrar en formato de tabla con encabezado y consecutivo

7. SALIR

- Finalice el programa.

Implementación

- Utiliza un arreglo de 2500 registros para almacenar los datos de los alumnos.
- Utilice una estructura para almacenar los datos básicos de un alumno:

None

```
struct Alumno {  
    int status;          // 1 = activo, 0 = eliminado  
    int matricula;  
    char apPat[50];  
    char apMat[50];  
    char nombre[50];  
    int edad;  
    char sexo;           // 'M' o 'F'  
};
```

- **Campo Llave es matricula**, por medio del cual se realizarán búsquedas y ordenación
- El programa debe estar estructurado de manera modular, separando la lógica en archivos **.h** y **.c** para facilitar su mantenimiento.
 - **Cree una librería propia que contenga funciones para:**
 - Manipulación de estructuras (struct).
 - Métodos de ordenación (burbuja, selección, inserción).
 - Métodos de búsqueda (secuencial y binaria).
 - Validación de datos de entrada.
 - **Utilice funciones para modularizar cada parte del proceso (agregar, eliminar, buscar, ordenar, imprimir).**
- Valide todas las entradas del usuario para evitar errores.
- Pruebe el programa con diferentes valores de entrada para asegurar su correcto funcionamiento.

- Menú Interactivo
 - Implementa un ciclo **do-while** que permita ejecutar cada opción del menú hasta que el usuario elija la opción de salida.
 - Maneja entradas inválidas con validaciones para evitar errores en la ejecución.

Formato del Reporte

El reporte debe incluir las siguientes secciones:

1. **Portada:** Nombre de la actividad, tu nombre, número de control y grupo.
2. **Introducción:** Breve descripción del propósito de la actividad.
3. **Teoría:** Explicación breve sobre estructuras (`struct`), métodos de ordenación y búsqueda.
4. **Código:** Incluye el bloque de código y salida más representativo de la actividad.
5. **Resultados:** Capturas de pantalla o texto con los resultados obtenidos al ejecutar el programa.
6. **Conclusiones:** Reflexión sobre lo aprendido y posibles mejoras.

NOTA: Código y salidas en documento anexo. especificar en el reporte.

Consideraciones Importantes

- Usa nombres descriptivos para variables, funciones y archivos.
 - Comenta tu código para facilitar su comprensión.
 - Valida las entradas del usuario para evitar errores.
 - Prueba tu programa con diferentes valores de entrada para asegurar su correcto funcionamiento.
 - Mantén una estructura clara y organizada en tu reporte.
-

Bibliografía y Ayuda

Las siguientes ligas proporcionan información relevante para la realización de esta actividad:

- [Programación en C: Uso de Estructuras](#)
- [Definición de nuevos nombres](#)
- <https://www.fing.edu.uy/tecnoinf/mvd/cursos/prinprog/material/teo/prinprog-teorico08.pdf>
- <https://kesquivel.files.wordpress.com/2013/05/estructuras2013final2.pdf>
- http://platea.pntic.mec.es/vgonzale/cyr_0204/cyr_01/control/lengua_C/estructuras.htm